



# Formal Relational Query Languages

**Database System Concepts, 6<sup>th</sup> Ed.**

©Silberschatz, Korth and Sudarshan

See [www.db-book.com](http://www.db-book.com) for conditions on re-use



# Formal Relational Query Languages

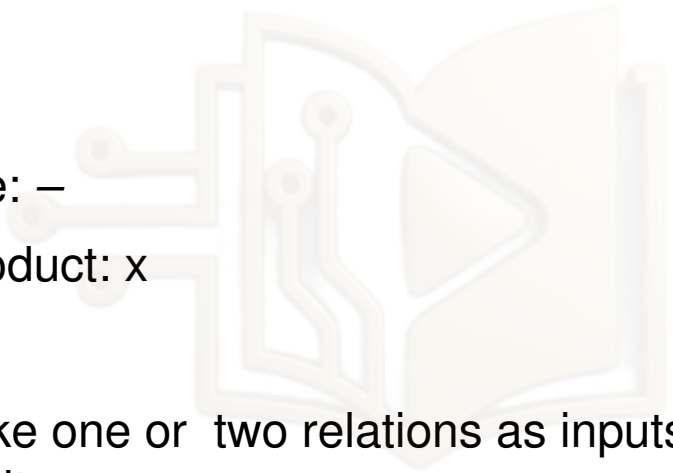
- Relational Algebra
- Tuple Relational Calculus
- Domain Relational Calculus





# Relational Algebra

- Procedural language
- Six basic operators
  - select:  $\sigma$
  - project:  $\Pi$
  - union:  $\cup$
  - set difference:  $-$
  - Cartesian product:  $\times$
  - rename:  $\rho$
- The operators take one or two relations as inputs and produce a new relation as a result.





# Select Operation – Example

■ Relation r

| A        | B        | C  | D  |
|----------|----------|----|----|
| $\alpha$ | $\alpha$ | 1  | 7  |
| $\alpha$ | $\beta$  | 5  | 7  |
| $\beta$  | $\beta$  | 12 | 3  |
| $\beta$  | $\beta$  | 23 | 10 |

■  $\sigma_{A=B \wedge D > 5}(r)$

| A        | B        | C  | D  |
|----------|----------|----|----|
| $\alpha$ | $\alpha$ | 1  | 7  |
| $\beta$  | $\beta$  | 23 | 10 |



# Select Operation

- Notation:  $\sigma_p(r)$
- $p$  is called the **selection predicate**
- Defined as:

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

Where  $p$  is a formula in propositional calculus consisting of **terms** connected by :  $\wedge$  (**and**),  $\vee$  (**or**),  $\neg$  (**not**)  
Each **term** is one of:

<attribute>     $op$  <attribute> or <constant>

where  $op$  is one of:  $=, \neq, >, \geq, <, \leq$

- Example of selection:

$$\sigma_{dept\_name="Physics"}(instructor)$$



# Project Operation – Example

■ Relation  $r$ :

| A        | B  | C |
|----------|----|---|
| $\alpha$ | 10 | 1 |
| $\alpha$ | 20 | 1 |
| $\beta$  | 30 | 1 |
| $\beta$  | 40 | 2 |

■  $\Pi_{A,C}(r)$

| A        | C |
|----------|---|
| $\alpha$ | 1 |
| $\alpha$ | 1 |
| $\beta$  | 1 |
| $\beta$  | 2 |

=

| A        | C |
|----------|---|
| $\alpha$ | 1 |
| $\beta$  | 1 |
| $\beta$  | 2 |



# Project Operation

- Notation:

$$\Pi_{A_1, A_2, \dots, A_k}(r)$$

where  $A_1, A_2$  are attribute names and  $r$  is a relation name.

- The result is defined as the relation of  $k$  columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets
- Example: To eliminate the *dept\_name* attribute of *instructor*

$$\Pi_{ID, name, salary}(instructor)$$



# Union Operation – Example

- Relations  $r, s$ :

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\alpha$ | 2   |
| $\beta$  | 1   |

$r$

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 2   |
| $\beta$  | 3   |

$s$

- $r \cup s$ :

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\alpha$ | 2   |
| $\beta$  | 1   |
| $\beta$  | 3   |



# Union Operation

- Notation:  $r \cup s$
- Defined as:

$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

- For  $r \cup s$  to be valid.
  1.  $r, s$  must have the **same arity** (same number of attributes)
  2. The attribute domains must be **compatible** (example: 2<sup>nd</sup> column of  $r$  deals with the same type of values as does the 2<sup>nd</sup> column of  $s$ )
- Example: to find all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or in both

$$\Pi_{\text{course\_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2009} (\text{section})) \cup \Pi_{\text{course\_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010} (\text{section}))$$



# Set difference of two relations

- Relations  $r, s$ :

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\alpha$ | 2   |
| $\beta$  | 1   |

$r$

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 2   |
| $\beta$  | 3   |

$s$

- $r - s$ :

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\beta$  | 1   |



# Set Difference Operation

- Notation  $r - s$
- Defined as:

$$r - s = \{t \mid t \in r \text{ and } t \notin s\}$$

- Set differences must be taken between **compatible** relations.
  - $r$  and  $s$  must have the **same** arity
  - attribute domains of  $r$  and  $s$  must be compatible
- Example: to find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$$\Pi_{course\_id}(\sigma_{semester="Fall" \wedge year=2009}(section)) - \Pi_{course\_id}(\sigma_{semester="Spring" \wedge year=2010}(section))$$



# Cartesian-Product Operation – Example

■ Relations  $r, s$ :

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\beta$  | 2   |

$r$

| $C$      | $D$ | $E$ |
|----------|-----|-----|
| $\alpha$ | 10  | a   |
| $\beta$  | 10  | a   |
| $\beta$  | 20  | b   |
| $\gamma$ | 10  | b   |

$s$

■  $r \times s$ :

| $A$      | $B$ | $C$      | $D$ | $E$ |
|----------|-----|----------|-----|-----|
| $\alpha$ | 1   | $\alpha$ | 10  | a   |
| $\alpha$ | 1   | $\beta$  | 10  | a   |
| $\alpha$ | 1   | $\beta$  | 20  | b   |
| $\alpha$ | 1   | $\gamma$ | 10  | b   |
| $\beta$  | 2   | $\alpha$ | 10  | a   |
| $\beta$  | 2   | $\beta$  | 10  | a   |
| $\beta$  | 2   | $\beta$  | 20  | b   |
| $\beta$  | 2   | $\gamma$ | 10  | b   |

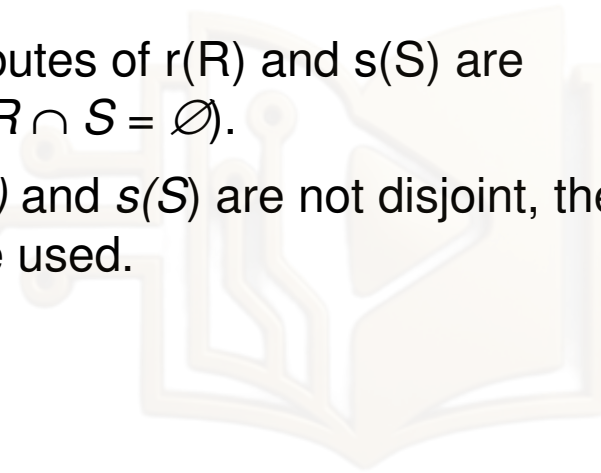


# Cartesian-Product Operation

- Notation  $r \times s$
- Defined as:

$$r \times s = \{t \ q \mid t \in r \textbf{ and } q \in s\}$$

- Assume that attributes of  $r(R)$  and  $s(S)$  are disjoint. (That is,  $R \cap S = \emptyset$ ).
- If attributes of  $r(R)$  and  $s(S)$  are not disjoint, then renaming must be used.





# Composition of Operations

- Can build expressions using multiple operations
- Example:  $\sigma_{A=C}(r \times s)$

■  $r \times s$

| A        | B | C        | D  | E |
|----------|---|----------|----|---|
| $\alpha$ | 1 | $\alpha$ | 10 | a |
| $\alpha$ | 1 | $\beta$  | 10 | a |
| $\alpha$ | 1 | $\beta$  | 20 | b |
| $\alpha$ | 1 | $\gamma$ | 10 | b |
| $\beta$  | 2 | $\alpha$ | 10 | a |
| $\beta$  | 2 | $\beta$  | 10 | a |
| $\beta$  | 2 | $\beta$  | 20 | b |
| $\beta$  | 2 | $\gamma$ | 10 | b |

■  $\sigma_{A=C}(r \times s)$

| A        | B | C        | D  | E |
|----------|---|----------|----|---|
| $\alpha$ | 1 | $\alpha$ | 10 | a |
| $\beta$  | 2 | $\beta$  | 10 | a |
| $\beta$  | 2 | $\beta$  | 20 | b |



# Rename Operation

- Allows us to name, and therefore to refer to, the results of relational-algebra expressions.
- Allows us to refer to a relation by more than one name.
- Example:

$$\rho_x(E)$$

returns the expression  $E$  under the name  $X$

- If a relational-algebra expression  $E$  has arity  $n$ , then

$$\rho_{x(A_1, A_2, \dots, A_n)}(E)$$

returns the result of expression  $E$  under the name  $X$ , and with the attributes renamed to  $A_1, A_2, \dots, A_n$ .



# Example Query

- Find the largest salary in the university
  - Step 1: find instructor salaries that are less than some other instructor salary (i.e. not maximum)
    - using a copy of *instructor* under a new name *d*
    - ▶  $\Pi_{instructor.salary} (\sigma_{instructor.salary < d.salary} (instructor \times \rho_d (instructor)))$
  - Step 2: Find the largest salary
    - ▶  $\Pi_{salary} (instructor) - \Pi_{instructor.salary} (\sigma_{instructor.salary < d.salary} (instructor \times \rho_d (instructor)))$



# Example Queries

- Find the names of all instructors in the Physics department, along with the *course\_id* of all courses they have taught

- Query 1

$$\Pi_{instructor.ID, course\_id} (\sigma_{dept\_name = \text{"Physics"}} (\sigma_{instructor.ID = teaches.ID} (instructor \times teaches)))$$

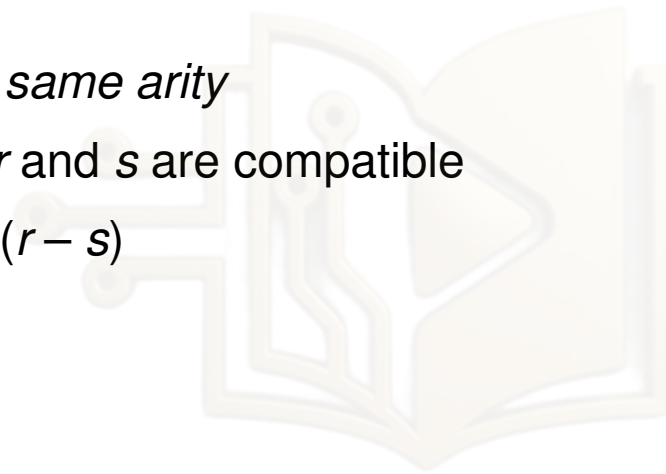
- Query 2

$$\Pi_{instructor.ID, course\_id} (\sigma_{instructor.ID = teaches.ID} (\sigma_{dept\_name = \text{"Physics"}} (instructor \times teaches)))$$



# Set-Intersection Operation

- Notation:  $r \cap s$
- Defined as:
- $r \cap s = \{ t \mid t \in r \textbf{ and } t \in s \}$
- Assume:
  - $r, s$  have the *same arity*
  - attributes of  $r$  and  $s$  are compatible
- Note:  $r \cap s = r - (r - s)$





# Set-Intersection Operation – Example

■ Relation  $r, s$ :

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\alpha$ | 2   |
| $\beta$  | 1   |

$r$

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 2   |
| $\beta$  | 3   |

$s$

■  $r \cap s$

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 2   |



# Natural-Join Operation

- Notation:  $r \bowtie s$
- Let  $r$  and  $s$  be relations on schemas  $R$  and  $S$  respectively.  
Then,  $r \bowtie s$  is a relation on schema  $R \cup S$  obtained as follows:
  - Consider each pair of tuples  $t_r$  from  $r$  and  $t_s$  from  $s$ .
  - If  $t_r$  and  $t_s$  have the same value on each of the attributes in  $R \cap S$ , add a tuple  $t$  to the result, where
    - ▶  $t$  has the same value as  $t_r$  on  $r$
    - ▶  $t$  has the same value as  $t_s$  on  $s$

- Example:

$R = (A, B, C, D)$

$S = (E, B, D)$

- Result schema =  $(A, B, C, D, E)$
- $r \bowtie s$  is defined as:

$$\Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$$



# Natural Join Example

■ Relations  $r$ ,  $s$ :

| $A$      | $B$ | $C$      | $D$ |
|----------|-----|----------|-----|
| $\alpha$ | 1   | $\alpha$ | a   |
| $\beta$  | 2   | $\gamma$ | a   |
| $\gamma$ | 4   | $\beta$  | b   |
| $\alpha$ | 1   | $\gamma$ | a   |
| $\delta$ | 2   | $\beta$  | b   |

$r$

| $B$ | $D$ | $E$        |
|-----|-----|------------|
| 1   | a   | $\alpha$   |
| 3   | a   | $\beta$    |
| 1   | a   | $\gamma$   |
| 2   | b   | $\delta$   |
| 3   | b   | $\epsilon$ |

$s$

■  $r \bowtie s$

| $A$      | $B$ | $C$      | $D$ | $E$      |
|----------|-----|----------|-----|----------|
| $\alpha$ | 1   | $\alpha$ | a   | $\alpha$ |
| $\alpha$ | 1   | $\alpha$ | a   | $\gamma$ |
| $\alpha$ | 1   | $\gamma$ | a   | $\alpha$ |
| $\alpha$ | 1   | $\gamma$ | a   | $\gamma$ |
| $\delta$ | 2   | $\beta$  | b   | $\delta$ |



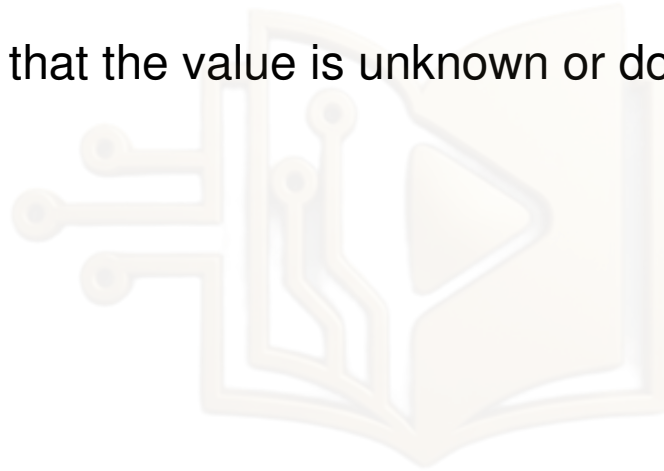
# Natural Join and Theta Join

- Find the names of all instructors in the Comp. Sci. department together with the course titles of all the courses that the instructors teach
  - $\Pi_{name, title} (\sigma_{dept\_name = \text{"Comp. Sci."}} (instructor \bowtie teaches \bowtie course))$
- Natural join is associative
  - $(instructor \bowtie teaches) \bowtie course$  is equivalent to  $instructor \bowtie (teaches \bowtie course)$
- Natural join is commutative
  - $instructor \bowtie teaches$  is equivalent to  $teaches \bowtie instructor$
- The **theta join** operation  $r \bowtie_{\theta} s$  is defined as
  - $r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$



# Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- Uses *null* values:
  - *null* signifies that the value is unknown or does not exist





# Outer Join – Example

## ■ Relation *instructor1*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> |
|-----------|-------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       |
| 12121     | Wu          | Finance          |
| 15151     | Mozart      | Music            |

## ■ Relation *teaches1*

| <i>ID</i> | <i>course_id</i> |
|-----------|------------------|
| 10101     | CS-101           |
| 12121     | FIN-201          |
| 76766     | BIO-101          |



# Outer Join – Example

## ■ Join

*instructor* ⋈ *teaches*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>course_id</i> |
|-----------|-------------|------------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       | CS-101           |
| 12121     | Wu          | Finance          | FIN-201          |

## ■ Left Outer Join

*instructor* ⋈<sub>L</sub> *teaches*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>course_id</i> |
|-----------|-------------|------------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       | CS-101           |
| 12121     | Wu          | Finance          | FIN-201          |
| 15151     | Mozart      | Music            | <i>null</i>      |



# Outer Join – Example

## ■ Right Outer Join

*instructor* ⋈<sub>r</sub> *teaches*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>course_id</i> |
|-----------|-------------|------------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       | CS-101           |
| 12121     | Wu          | Finance          | FIN-201          |
| 76766     | null        | null             | BIO-101          |

## ■ Full Outer Join

*instructor* ⋈<sub>f</sub> *teaches*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>course_id</i> |
|-----------|-------------|------------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       | CS-101           |
| 12121     | Wu          | Finance          | FIN-201          |
| 15151     | Mozart      | Music            | <i>null</i>      |
| 76766     | null        | null             | BIO-101          |



# Aggregate Functions and Operations

- **Aggregation function** takes a collection of values and returns a single value as a result.

**avg:** average value

**min:** minimum value

**max:** maximum value

**sum:** sum of values

**count:** number of values

- **Aggregate operation** in relational algebra

$$G_1, G_2, \dots, G_n \quad \mathcal{G} \quad F_1(A_1), F_2(A_2), \dots, F_n(A_n) (E)$$

$E$  is any relational-algebra expression

- $G_1, G_2, \dots, G_n$  is a list of attributes on which to group (can be empty)
- Each  $F_i$  is an aggregate function
- Each  $A_i$  is an attribute name

- Note: Some books/articles use  $\gamma$  instead of  $\mathcal{G}$  (Calligraphic G)



# Aggregate Operation – Example

■ Relation  $r$ :

| $A$      | $B$      | $C$ |
|----------|----------|-----|
| $\alpha$ | $\alpha$ | 7   |
| $\alpha$ | $\beta$  | 7   |
| $\beta$  | $\beta$  | 3   |
| $\beta$  | $\beta$  | 10  |

■  $G_{\text{sum}(c)}(r)$

| <b>sum(<math>c</math>)</b> |
|----------------------------|
| 27                         |



# Aggregate Operation – Example

- Find the average salary in each department

*dept\_name*    $\text{Avg}(\text{salary})$  (*instructor*)

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> |
|-----------|-------------|------------------|---------------|
| 76766     | Crick       | Biology          | 72000         |
| 45565     | Katz        | Comp. Sci.       | 75000         |
| 10101     | Srinivasan  | Comp. Sci.       | 65000         |
| 83821     | Brandt      | Comp. Sci.       | 92000         |
| 98345     | Kim         | Elec. Eng.       | 80000         |
| 12121     | Wu          | Finance          | 90000         |
| 76543     | Singh       | Finance          | 80000         |
| 32343     | El Said     | History          | 60000         |
| 58583     | Califieri   | History          | 62000         |
| 15151     | Mozart      | Music            | 40000         |
| 33456     | Gold        | Physics          | 87000         |
| 22222     | Einstein    | Physics          | 95000         |

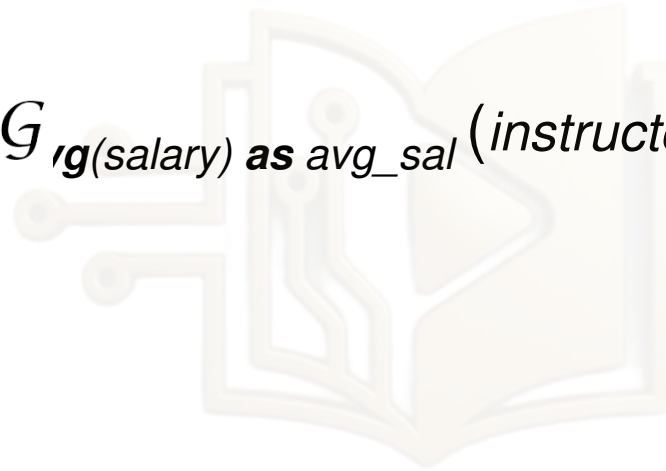
| <i>dept_name</i> | <i>avg_salary</i> |
|------------------|-------------------|
| Biology          | 72000             |
| Comp. Sci.       | 77333             |
| Elec. Eng.       | 80000             |
| Finance          | 85000             |
| History          | 61000             |
| Music            | 40000             |
| Physics          | 91000             |



# Aggregate Functions (Cont.)

- Result of aggregation does not have a name
  - Can use rename operation to give it a name
  - For convenience, we permit renaming as part of aggregate operation

*dept\_name*      $G_{avg(salary) \text{ as } avg\_sal}(instructor)$





# SQL and Relational Algebra

- **select**  $A_1, A_2, \dots, A_n$   
**from**  $r_1, r_2, \dots, r_m$   
**where**  $P$

is equivalent to the following expression in multiset relational algebra

$$\Pi_{A_1, \dots, A_n} (\sigma_P (r_1 \times r_2 \times \dots \times r_m))$$

- **select**  $A_1, A_2, \text{sum}(A_3)$   
**from**  $r_1, r_2, \dots, r_m$   
**where**  $P$   
**group by**  $A_1, A_2$

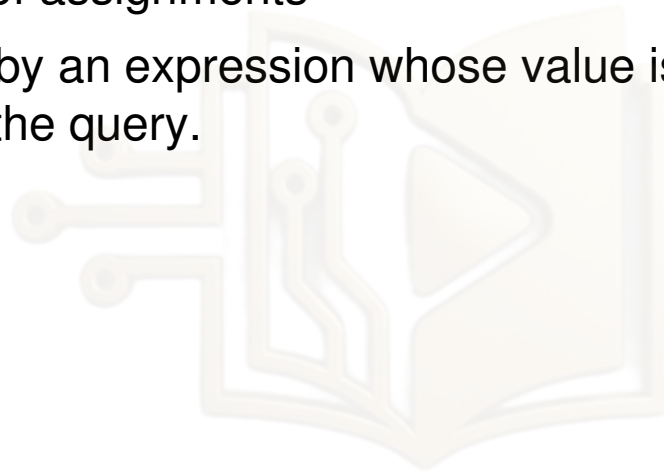
is equivalent to the following expression in multiset relational algebra

$$A_1, A_2 \quad \mathcal{G}_{n(A_3)} (\sigma_P (r_1 \times r_2 \times \dots \times r_m))$$



# Assignment Operation

- The assignment operation ( $\leftarrow$ ) provides a convenient way to express complex queries.
- Write query as a sequential program consisting of
  - ▶ a series of assignments
  - ▶ followed by an expression whose value is displayed as a result of the query.



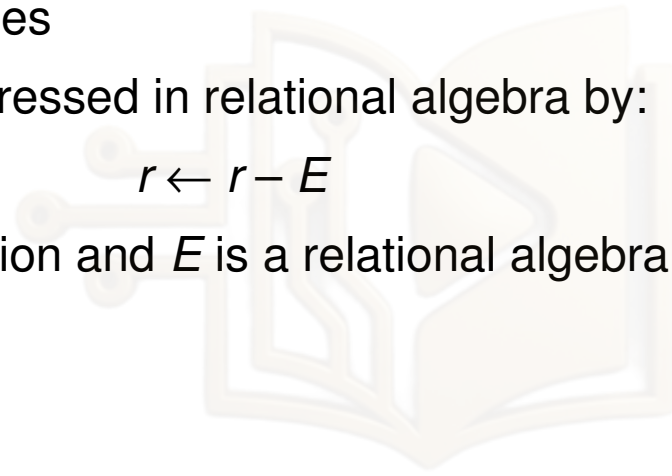


# Deletion

- A delete request is expressed similarly to a query, except instead of displaying tuples to the user, the selected tuples are removed from the database.
- Can delete only whole tuples; cannot delete values on only particular attributes
- A deletion is expressed in relational algebra by:

$$r \leftarrow r - E$$

where  $r$  is a relation and  $E$  is a relational algebra query.





# Deletion Examples

- Delete all account records in the Perryridge branch.

$account \leftarrow account - \sigma_{branch\_name = "Perryridge"}(account)$

- Delete all loan records with amount in the range of 0 to 50

$loan \leftarrow loan - \sigma_{amount \geq 0 \text{ and } amount \leq 50}(loan)$

- Delete all accounts at branches located in Needham.

$r_1 \leftarrow \sigma_{branch\_city = "Needham"}(account \bowtie branch)$

$r_2 \leftarrow \Pi_{account\_number, branch\_name, balance}(r_1)$

$r_3 \leftarrow \Pi_{customer\_name, account\_number}(r_2 \bowtie depositor)$

$account \leftarrow account - r_2$

$depositor \leftarrow depositor - r_3$



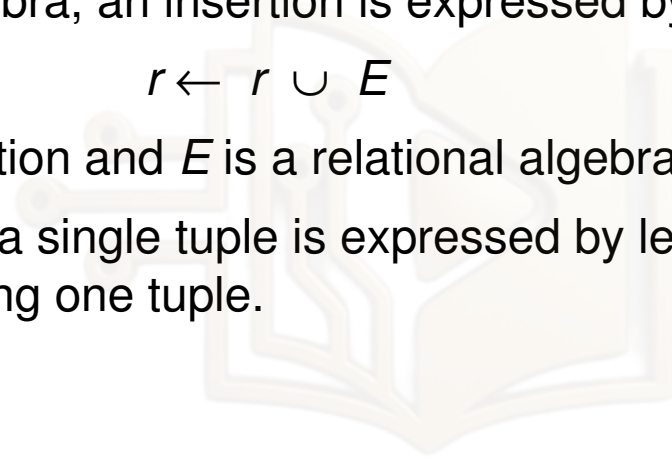
# Insertion

- To insert data into a relation, we either:
  - specify a tuple to be inserted
  - write a query whose result is a set of tuples to be inserted
- in relational algebra, an insertion is expressed by:

$$r \leftarrow r \cup E$$

where  $r$  is a relation and  $E$  is a relational algebra expression.

- The insertion of a single tuple is expressed by letting  $E$  be a constant relation containing one tuple.





# Insertion Examples

- Insert information in the database specifying that Smith has \$1200 in account A-973 at the Perryridge branch.

$$account \leftarrow account \cup \{("A-973", "Perryridge", 1200)\}$$
$$depositor \leftarrow depositor \cup \{("Smith", "A-973")\}$$

- Provide as a gift for all loan customers in the Perryridge branch, a \$200 savings account. Let the loan number serve as the account number for the new savings account.

$$r_1 \leftarrow (\sigma_{branch\_name = "Perryridge"}(borrower \bowtie loan))$$
$$account \leftarrow account \cup \Pi_{loan\_number, branch\_name, 200}(r_1)$$
$$depositor \leftarrow depositor \cup \Pi_{customer\_name, loan\_number}(r_1)$$



# Updating

- A mechanism to change a value in a tuple without changing *all* values in the tuple
- Use the generalized projection operator to do this task

$$r \leftarrow \Pi_{F_1, F_2, \dots, F_l}(r)$$

- Each  $F_i$  is either
  - the  $i^{\text{th}}$  attribute of  $r$ , if the  $i^{\text{th}}$  attribute is not updated, or,
  - if the attribute is to be updated  $F_i$  is an expression, involving only constants and the attributes of  $r$ , which gives the new value for the attribute



# Update Examples

- Make interest payments by increasing all balances by 5 percent.

$account \leftarrow \Pi_{account\_number, branch\_name, balance * 1.05}(account)$

- Pay all accounts with balances over \$10,000 6 percent interest and pay all others 5 percent

$account \leftarrow \Pi_{account\_number, branch\_name, balance * 1.06}(\sigma_{BAL > 10000}(account))$   
 $\cup \Pi_{account\_number, branch\_name, balance * 1.05}(\sigma_{BAL \leq 10000}(account))$



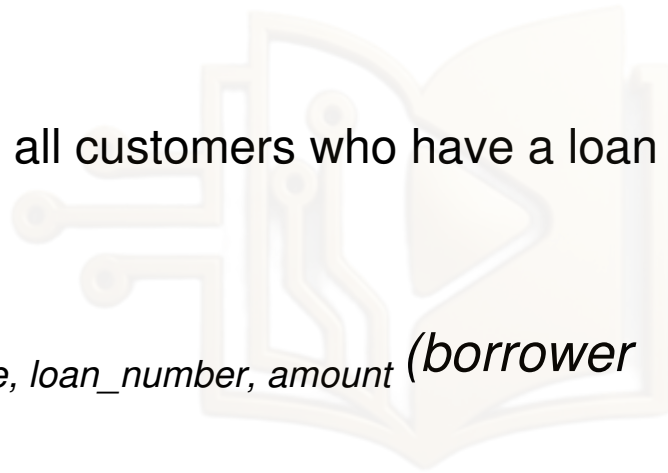
# Example Queries

- Find the names of all customers who have a loan and an account at bank.

$$\Pi_{customer\_name} (borrower) \cap \Pi_{customer\_name} (depositor)$$

- Find the name of all customers who have a loan at the bank and the loan amount

$$\Pi_{customer\_name, loan\_number, amount} (borrower \bowtie loan)$$





# Example Queries

- Find all customers who have an account from at least the “Downtown” and the Uptown” branches.

- Query 1

$$\Pi_{customer\_name} (\sigma_{branch\_name = \text{“Downtown”}} (depositor \bowtie account)) \cap \Pi_{customer\_name} (\sigma_{branch\_name = \text{“Uptown”}} (depositor \bowtie account))$$

- Query 2

$$\Pi_{customer\_name, branch\_name} (depositor \bowtie account) \div \rho_{temp(branch\_name)} (\{(\text{“Downtown”}), (\text{“Uptown”})\})$$

Note that Query 2 uses a constant relation.



# Bank Example Queries

- Find all customers who have an account at all branches located in Brooklyn city.

$$\Pi_{customer\_name, branch\_name} (depositor \bowtie account) \\ \div \Pi_{branch\_name} (\sigma_{branch\_city = \text{"Brooklyn"}} (branch))$$

